ADK Capstone eComBot

- This document explains the fully integrated eComBot capstone implementation.
- Deterministic mode runs without paid model keys; live ADK, LiteLLM, and LangSmith modes are
- environment-controlled.

Implemented Modules

- Day 01-02: support agent foundation, prompt variants, and domain boundaries.
- Day 03: order tools and in-memory session state for customer name, last order, and last
- product.
- Day 04: active Redis session persistence, PostgreSQL business tools, and durable turn
- history.
- Day 05-06: LangChain Runnable RAG over product, FAQ, and PDF records stored in ChromaDB.
- Day 07: real LiteLLM completion routing with primary and fallback providers.
- Day 08: real FastMCP client/server order and inventory tool execution.
- Day 09: Sales Agent ReAct reasoning and reflection after rejected recommendations.
- Final hardening: Google ADK multi-agent tree, LangSmith tracing, 12 Promptfoo cases, CI/CD,
- guardrails, voice adapters, and Chainlit UI.

Agents

- Support Agent handles order status, cancellation, invoices, inventory, product lookup, and
- grounded policy answers.
- Sales Agent handles recommendations, comparisons, ReAct reasoning, and reflection.
- Orchestrator routes between support and sales, applies guardrails, filters output, and
- records traces.

Tools and Backend Calls

- Order tools: get_order_status, cancel_order, and get_invoice validate order IDs and return
- structured data.
- Product tools: lookup_product, check_stock, list_variants, and catalog filtering use the
- local product corpus.
- FastMCP client invokes a real in-process or HTTP FastMCP server with structured results.

RAG and PDF Ingestion

- Products and FAQ are stored as rich JSON source documents.
- PDF knowledge documents are extracted, chunked with overlap, tagged with source file, title,
- section, page, and doc_type metadata, and written to document_index.json.
- LangChain RunnableLambda and StructuredTool components query ChromaDB using deterministic
- local embeddings and confidence filtering.

Model Routing

- The gateway route classifier chooses fast-faq for simple support traffic and deep-support
- for comparisons, recommendations, complaints, and multi-step decisions.
- LiteLLM performs real completion calls and provider fallback in live mode; the proxy config

- supports OpenAI primary and OpenRouter fallback.

Security and Observability

- Input guardrail blocks prompt injection, role override, system prompt extraction, and data
- exfiltration attempts.
- Output guardrail redacts email addresses, phone numbers, and unsupported competitor
- references without redacting delivery dates.
- LangSmith traceable runs capture agent execution when configured; JSONL tracing remains
- always available locally.
- Promptfoo evaluates 12 support, sales, RAG, MCP, reflection, and security cases through the
- same FastAPI runtime.

How to Execute Locally

- Install base dependencies: python -m pip install -r requirements.txt
- Build local RAG index: python -m src.rag.embed_catalog
- Run scenario: python run_agent.py --scenario
- Run evals: python run_agent.py --eval
- Run tests: python -m pytest -q
- Run Chainlit UI: chainlit run src/ui/app.py -w
- Run API: python -m uvicorn src.api.app:app --port 8001
- Run Promptfoo: npm run eval
- Run Rancher stack: docker compose up --build -d